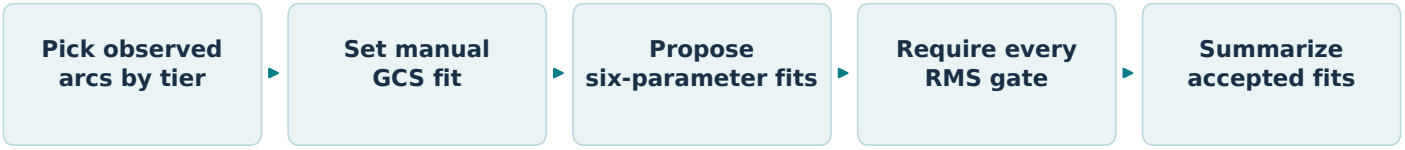
How GCS uncertainty is calculated

A code-based explanation of the current two-tier GUI and search engine

The program searches for alternative GCS geometries compatible with your traced boundaries, relative to your manual fit. The spread of the retained fits is the reported uncertainty description.



The four requested answers

1. Tier 1 and Tier 2 label observed boundary sections by your confidence. Both use the same RMS distance calculation, with a separate acceptance check for every traced view and tier. Fresh extra-RMS defaults are +10% and +20%, respectively. See pages 2–4.

2. Manual fit sets the six-parameter reference geometry. It centers the search box and sets each component’s frozen baseline RMS. Tracing supplies the observed constraints; manual fitting supplies the model reference. See page 5.

3. Broad candidates use scrambled six-dimensional Sobol sampling. The mixed sampler replaces selected angular offsets with signed historical fit-to-fit differences plus Gaussian jitter. It then rejects invalid or incompatible candidates. See pages 6–7 and 9.

4. History-guided and broad percentages choose the proposal mix. At 100% broad, all proposals use the broad Sobol design. At 100% history, shape parameters, missing tilt and failed historical retries can still be broad. Neither setting changes the RMS acceptance rule. See page 8.

What the result means

The accepted ensemble produces min/max values, a median, empirical percentiles and histograms. These are proposal-dependent compatibility ranges. The software does not calibrate them as confidence intervals or estimate an automatic Gaussian standard error. Details and exported files are on pages 10–11; a source navigation index is on page 12.

Scope: current local source reviewed on 31 October 2026; Git revision b263ea338f19. The active import is gui.py → TieredPanel, which inherits shared search machinery from uncertainty_gui.py. Legacy branches are distinguished on page 11.

Code locations (paths relative to the project root):

src/gui.py:29; src/tiered_gui.py:16–21, 458–479; src/uncertainty_gui.py:134–272; src/outline_uncertainty.py:129–141.

What Tier 1 and Tier 2 actually do

Observed boundary annotations become separate geometric constraints

Input	Meaning	Fresh extra RMS
Tier 1 / cyan	Clear, more confidently identified boundary sections	10% of its own manual RMS
Tier 2 / yellow	Less certain boundary sections	20% of its own manual RMS

You choose the confidence category. The code does not infer a tier from image brightness, assign a probability to a line, or move a picked line randomly. A mouse click records a native image coordinate in a list belonging to one view, one tier and one arc.

Why separate arcs matter

An arc is an open polyline through your points. Use a new arc when an observed boundary disappears or has a gap. The numerical calculation resamples each arc separately and never draws a connecting constraint across that gap. Nonempty arcs need at least two distinct points; an incomplete one-point arc prevents scoring.

Assigning a segment to the other tier preserves its endpoint coordinates. The segment is removed from its source arc and becomes an arc in the destination tier; adjacent segments are preserved. This changes which RMS pool and allowance apply to that observed edge.

How tiers affect acceptance

Every populated view/tier is one component. All arcs within that component contribute to a single arclength-weighted RMS. Each component must pass its own threshold. There is no combined Tier 1/Tier 2 score in which a good match can compensate for a failure elsewhere. A view or tier with no trace adds no acceptance check.

> Tier 2 is intended to be more permissive, but both tiers use exactly the same distance formula. Their separate manual baselines also differ: +20% of Tier 2’s baseline need not exceed +10% of Tier 1’s baseline in absolute RSun. Current percentage mode lets you edit the two percentages independently.

A component’s aggregate RMS is tested, rather than a maximum pointwise distance or a separate condition for every arc. Some local errors can therefore be larger than the RMS threshold while the component still passes.

Code locations (paths relative to the project root):

src/tiered_gui.py:22–24, 164–172, 199–258; src/tiered_outlines.py:28–43, 76–94, 103–128, 131–141; defaults: src/tiered_outlines.py:7–11. Tier ordering at :81–82 applies only to legacy absolute allowances.

The exact outline RMS calculation

The same measurement is applied to the manual fit and every candidate

1. Generate the candidate’s exterior boundary

The six GCS parameters generate a three-dimensional surface mesh. SunPy transforms it into each image’s observer coordinate frame and WCS pixels. The scoring mesh starts at 10 straight, 24 front and 48 circle vertices, multiplied by mesh density. It is independent of the displayed green wireframe.

Surface quads become triangles. Their projected union is rasterized; enclosed holes are filled and the exterior boundary is extracted. Concavity is preserved. Crop-edge pixels are excluded so an image border cannot masquerade as a CME edge. Default raster resolution is 1024 along the longest image dimension.

2. Convert image coordinates into projected solar radii

$$(X, Y) = \frac{D}{R_\odot}\left(\tan T_x, \frac{\tan T_y}{\cos T_x}\right)$$

WCS supplies helioprojective angles Tx and Ty in radians. D is the image’s DSUN_OBS in metres; RSun is the fixed nominal solar radius, 695,700 km. This intersects the viewing ray with the plane through the Sun perpendicular to the observer–Sun line. Both traces and model boundary points use these projected coordinates; these are not deprojected three-dimensional distances.

3. Resample arcs, find nearest boundary points, calculate RMS

After physical conversion, each arc is resampled at 128 equally spaced arclength locations. An arc of length L has weights L/127 at interior samples and L/254 at the endpoints. Its weights sum to L, so longer arcs contribute more; additional clicks alone do not increase its weight.

$$d_i(p) = \min_{b \in B_v(p)} \|q_i - b\|, \qquad E_{v,t}(p) = \sqrt{\frac{\sum_i w_i d_i(p)^2}{\sum_i w_i}}$$

Here q_i is an observed sample, B_v(p) is the raster exterior in that view, and the sum pools arcs in one view/tier. cKDTree finds the nearest sampled boundary point. Distances go from observed arcs to the model only: untraced model sections incur no missing-arc penalty. Finite mesh/raster resolution can affect a close pass or failure.

Code locations (paths relative to the project root):

src/search_scoring.py:15–17, 51–70; src/outline_uncertainty.py:26–39, 49–92; src/distance_metric.py:22–57; src/tiered_outlines.py:28–49, 52–91, 103–120. Raster default: src/uncertainty_gui.py:421–429.

Exactly which alternative fits pass

A separate, frozen manual-relative gate for each traced view and tier

At the start of a search the program projects the current manual fit p_0 and calculates $E_0(v,t)$, its RMS for each populated component. For an extra-RMS allowance a(v,t), a candidate p must satisfy:

$$E_{v,t}(p) \le E_{v,t}(p_0)\left(1+\frac{a_{v,t}}{100}\right) \quad \text{for every populated } (v,t)$$

The baselines stay fixed for the entire run. Finding a better candidate does not lower later thresholds. There is no absolute error floor or extra absolute cap in the current rule.

Worked example — every RMS is in projected RSun

Component	Manual RMS	Extra	Allowed RMS	Fit A	Fit B
STA / Tier 1	0.050	10%	0.055	0.054	0.056
STA / Tier 2	0.100	20%	0.120	0.115	0.030
STB / Tier 1	0.040	10%	0.044	0.043	0.020

Fit A passes all three components and is accepted. Fit B fails STA/Tier 1 because 0.056 > 0.055. Its excellent agreement in the other two components cannot rescue it. A fourth untraced view would add visual context without another gate.

Boundary cases

Extra RMS = 0%: candidates must be at least as good as the manual fit in every component. Different parameter vectors can still pass; the manual vector is not the only possible solution.

Extra RMS = 100%: allowed RMS is twice that component's manual RMS. This is a tolerance setting and is separate from 100% history-guided or 100% broad search.

Manual RMS = 0: allowed RMS remains 0 at any finite percentage. There is no small positive floor added to make sampling easier. Equality at the threshold is accepted.

For the same percentage, a poorer manual match gives a larger absolute allowed mismatch. The quality of the manual reference therefore directly affects the resulting compatible range.

Code locations (paths relative to the project root):

src/uncertainty_gui.py:140–146 (freeze manual scores); src/tiered_outlines.py:22–25, 124–128 (threshold and all-component test); src/search_scoring.py:73–99 (candidate evaluation).

The exact role of the manual fit

It supplies a reference model; tier picking supplies observed boundary data

Code key	Geometric role	Units
lat	Heliographic latitude	degrees
lon	Stonyhurst longitude	degrees; modulo 360
tilt	Geometric orientation of the flux rope	degrees; modulo 180
height	Apex height	solar radii
half_angle	GCS half-angle	degrees
kappa	GCS aspect ratio κ	dimensionless

The Manual fit sliders change these six numbers and redraw the projected three-dimensional model. They do not change your traced observed lines. Conversely, picking or reclassifying a line changes the data constraints without solving for the six parameters automatically.

The manual vector p_0 has four concrete uses

1. Proposal center: every candidate starts from p_0 plus a bounded offset. The editable scales and multiplier set the half-width of that box; historical proposals are also offsets about this same manual vector.

2. Acceptance reference: the worker calculates the manual RMS for each view/tier once. Those scores set all allowed candidate RMS values for that run.

3. Angular reference: saved offsets keep longitude and tilt continuous around the manual orientation when summaries and histograms are calculated.

4. Display and persistence: the manual fit is the bright pink search overlay and dashed red histogram marker. It is saved separately as manual_fit.json and is not inserted as an artificial accepted draw.

Moving the manual fit changes both the explored region and the baseline-relative gates. The search does not optimize or declare the manual fit to be a global minimum. Changing the fit, traces or relevant settings clears current search results; fitting and tracing controls are disabled during a run.

Code locations (paths relative to the project root):

src/gui.py:621–637, 780–833, 849–857; src/tiered_gui.py:287–306, 458–479; src/uncertainty_gui.py:140–146, 549–567, 595–610, 1013–1020; src/outline_uncertainty.py:101–105, 129–141; src/session.py:95–97.

How broad candidate sampling works

Scrambled low-discrepancy Sobol points in a bounded six-dimensional box

The broad sampler calls scipy.stats.qmc.Sobol(d=6, scramble=True, seed=seed).random_base2(exponent). It creates N = 2^m rows in the unit cube, where m is the exponent. Each coordinate is transformed as follows:

$$h_j = s_j M, \qquad \Delta_j = (2u_j - 1)h_j, \qquad p_j = p_{0,j} + \Delta_j$$

s is the editable scale and M is the multiplier. Broad proposals explore the box uniformly by design, using a scrambled Sobol pattern for coverage. They are not a six-parameter Gaussian draw, an MCMC chain, a grid of every combination or an optimization sequence.

Fresh controls and scale meanings

Parameter group	How scale is initialized	Multiplier 2 implies
Latitude, longitude, tilt	Eligible-history mean eventwise maximum absolute pair difference; controls round to 0.1°	Half-width = 2 × the displayed angular scale
Height	Exploratory scale 0.6 RSun	±1.2 RSun
Half-angle	Exploratory scale 5.0°	±10.0°
κ	Exploratory scale 0.05	±0.10

The historical-difference column is a reference value; editing the scale changes the range. These scales are not automatically inferred one-sigma errors. If history cannot load, angular fallback constants are 5.74°, 11.23° and 24.71°. Saved sessions can restore other scales.

Wrapping, fixed parameters and rejected geometry

Longitude is wrapped modulo 360°; tilt is wrapped into [−90°, 90°). Unwrapped offsets are retained. A zero scale fixes that parameter; at least one of the six scales must be nonzero. Longitude half-width may not exceed 180° and tilt half-width may not exceed 90°, avoiding repeated equivalent orientations.

The sampler discards nonfinite rows and geometries outside |latitude| ≤ 90°, height > 0, 0° < half-angle < 90°, and 0 < κ < 1. Geometries with unusable projections are also rejected during scoring. Thus the valid or accepted population need not fill the original box uniformly.

Code locations (paths relative to the project root):

src/outline_uncertainty.py:14–23, 101–126; src/historical_sampling.py:17–25; src/uncertainty_gui.py:369–420; src/search_scoring.py:73–99.

How historical guidance works

A mixture of broad Sobol rows and smoothed empirical angular differences

The historical sampler first generates the same full broad Sobol array. With broad fraction f, it randomly chooses round[N × (1 − f)] row indices for attempted angular replacement. This count is zero if all three angular half-widths are zero.

For each chosen row, up to 64 attempts

1. Choose an eligible historical event uniformly, then choose one available fit pair uniformly within that event. Events with more pairs do not receive a larger initial event-selection probability.

2. Choose a common sign +1 or −1 with equal probability. Apply it jointly to the available latitude, longitude and tilt differences, preserving their joint sign structure. For each available nonfixed angular coordinate:

$$\Delta_j = \eta\, d_{e,k,j} + \varepsilon_j, \quad \eta \in \{-1, +1\}, \quad \varepsilon_j \sim N(0, (0.10 s_j)^2)$$

The Gaussian jitters are independent by coordinate. Their standard deviation is 10% of the scale, not 10% of scale × multiplier. Full historical fit-to-fit differences are used; there is no division by √2 to estimate a single-fit error.

3. A zero-width angular coordinate stays fixed. If the chosen pair lacks tilt, its original broad Sobol tilt is retained. Height, half-angle and κ always retain their original broad values.

4. Reject and retry if an angular offset exceeds its configured half-width (apart from a numerical tolerance of 10^{-12}). Offsets are not clipped. If all 64 attempts fail, retain the original complete broad row. Then wrap angles and apply the normal geometry filters.

Which historical information enters

The local JSON contains curated differences from 56 events and 137 fits, with signed latitude, shortest longitude and 180°-periodic geometric tilt differences. Missing tilt remains missing. The loader excludes review-flagged events and the target calendar date. Currently cases 3, 29 and 52 are flagged; July 12, 2012 is case 52. No eligible record of that target is used to guide its own search.

For each angular scale, the initializer takes the mean across eligible events of that event’s largest absolute available pair difference. Bounds rejection can change historical event frequencies. The code applies no importance correction or likelihood weighting afterward.

Code locations (paths relative to the project root):

src/historical_sampling.py:17–25 (scales), 28–54 (load/exclusions), 57–121 (mixture, jitter, retry, labels); src/historical_gcs_proposals.json:1–9 (format and provenance); src/uncertainty_gui.py:58–66 (dispatch).

History-guided and broad at 100%

The controls change where fits are proposed, while the same acceptance checks apply

The two visible percentages are linked and sum to 100%. Fresh defaults are history-guided 70% / broad 30%. The requested mixture describes proposal generation, not the accepted shares or the probability that the true CME lies in a range.

Setting	What the code actually does
Broad 100% / history 0%	Switches to the uniform sampling mode. All six offsets come from the bounded scrambled Sobol design. No historical vectors or historical Gaussian jitter are used. Initial angular scales can still have been initialized from history.
History 100% / broad 0%	Selects every row for attempted historical angular replacement, if any angular width is positive. Height, half-angle and κ remain broad. Missing tilt remains broad, and 64 failed retries leave a complete broad row.
Mixed, e.g. history 70% / broad 30%	Randomly selects approximately the requested share of rows for angular replacement. The other rows remain broad. Bounds, invalid geometry, outline gates and stopping change the eventual tried and accepted shares.

What stays the same at either endpoint

The manual center, all six configured half-widths, geometry validity tests, frozen per-view/tier manual baselines and extra-RMS acceptance thresholds are unchanged. “100% broad” does not mean searching the entire physical GCS parameter space: it still searches only your configured box.

Important limiting cases

With all angular scales zero, even 100% history uses broad Sobol sampling for any varying shape parameters. If eligible history is unavailable when the panel initializes, the controls are forced to broad-only.

Even with history = 100%, a row labeled history-direction has broad tilt because the historical pair lacks it. A row labeled broad can be a bounded-retry fallback. Source labels and historical case numbers are exported for accepted rows.

The separate Extra RMS = 100% setting means RMS ≤ 2 × manual RMS. It does not change the proposal mix. History/broad settings at 100% do not relax any RMS gate.

Code locations (paths relative to the project root):

src/uncertainty_gui.py:323–337, 374–382, 580–593; src/historical_sampling.py:73–119; src/tiered_outlines.py:22–25; accepted labels: src/uncertainty_gui.py:270–271, 1127–1137.

How fits are tried and retained

Batch generation, frozen scoring, deterministic ordering and stopping

Fresh default	Value	Where
Sobol batch size	2^{12} = 4,096 generated proposals	uncertainty_gui.py:413–416
Initial seed	2022	uncertainty_gui.py:417–420
Accepted target	512 distinct fits	uncertainty_gui.py:481–486
Generated-proposal budget	262,144 (64 default batches)	uncertainty_gui.py:26; tiered_gui.py:468
CPU processes	Up to 4 initially; selectable	uncertainty_gui.py:476–479

Restored sessions can replace these defaults. Batch size, seed and scoring resolution are internal controls in the compact interface; settings are saved with the run.

The actual run sequence

1. Read the current reference and traces; prepare the components; project the manual fit and freeze its RMS scores. Each new batch uses seed (initial seed + batch index) modulo 2^{32}. Batches are newly scrambled Sobol designs, not continuations of one balanced Sobol sequence.

2. Generate candidates through the selected broad or mixed design. Filter invalid geometry and remove exact duplicate six-parameter tuples across batches. Score remaining candidates in proposal order.

3. Project each traced view and evaluate its components. After a failed view the scorer can skip later views. Every retained accepted fit has every component score. Untraced views are projected separately for display and cannot reject a fit.

4. Keep the first requested number of accepted distinct fits in candidate order. Stop on the target, cancellation or the generated-proposal budget. Partial results are retained. The bounds and acceptance thresholds do not adapt between batches.

CPU workers return results in proposal order, so faster-to-score geometries get no preference. Later accepted fits already in flight after the target are counted separately and excluded from the ensemble. The number tested can vary slightly with CPU count. A failed CPU pool resumes sequentially from the unprocessed suffix.

> “Proposed” counts full generated batches. “Tested” counts scored rows. Invalid geometries, duplicates, unevaluated batch tails and in-flight work explain differences. Filtering and stopping mean the accepted ensemble is not itself a balanced Sobol design.

Code locations (paths relative to the project root):

src/uncertainty_gui.py:110–132, 134–272; src/search_scoring.py:33–48, 73–99, 132–171; configuration: src/tiered_gui.py:458–479.

How the accepted spread is calculated

All retained accepted geometries contribute equally to the summaries

Let A be the retained accepted rows, and N_A their count. For each parameter the code takes the actual accepted values and computes NumPy percentiles at 2.5, 16, 50, 84 and 97.5, together with the observed minimum and maximum. The 50th percentile is the median. An empty accepted set has an empty summary.

Saved field	Calculation / meaning
min, max	Smallest and largest retained accepted values
p2_5, p16, median, p84, p97_5	np.percentile(samples, [2.5, 16, 50, 84, 97.5])
search_edges	Flags a parameter if any accepted |offset| ≥ 0.95 × its positive half-width
accepted_offset_covariance	np.cov(offsets, rowvar=False), when N_A > 1

Longitude and tilt use manual value + saved unwrapped offset, rather than the wrapped stored angle. For example, a longitude cloud around 359° can summarize as 358°–361° without an artificial split at 0°/360°. The parameter summary and histogram use the same convention.

Histogram fractions

$$\text{bar fraction} = \frac{\text{number of accepted rows in bin}}{N_A}$$

Each fit has equal counting weight. Bin fractions sum to one for each parameter. The usual bin count is min(40, max(5, ceil(√N_A))); identical values use one bin. The dashed red marker is the manual value. There is no weighting by RMS quality or proposal source.

How to read these numbers scientifically

p16–p84 and p2.5–p97.5 are empirical central percentile spans of the accepted proposal population. They are not calibrated 68% or 95% confidence/credible intervals. More accepted fits improve the description of this chosen search but do not remove its dependence on traces, manual fit, allowances, bounds and mixture.

Arclength weights are used inside an outline RMS; they are not statistical weights on the accepted geometries. The exported covariance describes the accepted offsets only. Preserve whole six-parameter rows when using the ensemble: separate marginal ranges do not imply every combination is compatible.

Code locations (paths relative to the project root):

src/outline_uncertainty.py:129–141; src/parameter_histograms.py:48–81; src/uncertainty_gui.py:1111–1121; interpretation: src/tiered_gui.py:478–479, 555.

Saved results and older code paths

Use each run’s saved units, allowance mode and configuration

Output	What it contains
manual_fit.json / manual_traces.json	User reference; native-pixel arcs and tier assignments; percentage allowances, image identities and settings.
search.json	Configuration, manual/allowed RMS values, component definitions, units, stop reason, batch seeds, diagnostics and sampling metadata.
accepted_all.csv / selected_fits.csv	Current retained accepted vectors, their component RMS scores, proposal source and historical case. In current target mode the selected ensemble is the retained first N.
parameter_summary.csv / accepted_arrays.npz	Marginal summaries; arrays of accepted parameters, offsets, errors and selected indices.
accepted_parameter_histograms.png	Fractions from all retained accepted fits; exported along with fit/outline figures.

The launcher stores frames under sessions/<event>/frames/<UTC time>/ and each search in a new gcs_uncertainty_* subfolder. time_series.csv collects manual parameters and search status. Times are fitted independently: a new time can inherit fit/settings, but requires fresh native-pixel outlines. This workflow does not calculate speed errors or propagate to magnetic-field/Bz forecasts.

Why older files or functions may look different

The active GUI uses TieredPanel, although the base single-outline UncertaintyPanel remains in the source. Latest annotations are gcs-outlines-v4 with projected-solar-radius scores and percent allowances. Untagged low-level configurations retain their old native-pixel score units and additive threshold: allowed RMS = manual RMS + absolute allowance.

Loading older annotations can import a single outline as Tier 1 and convert an additive allowance to 100 × absolute allowance / manual RMS. That RMS is recomputed for the saved manual fit and scoring settings. Conversion preserves the additive threshold for the prepared tiered metric; old single-outline scoring used unweighted RMS. Pixel migration requires a validated similarity WCS. Positive additive allowance at zero baseline cannot be converted. Restored old results keep their original rule and rows.

The README’s illustrated older run used +0.10 / +0.20 RSun additive allowances. Those values are not today’s fresh defaults of +10% / +20%. Saved sessions can legitimately restore percentages converted from older absolute settings.

Code locations (paths relative to the project root):

src/session.py:34–56, 79–130; src/timeseries.py:54–67; src/uncertainty_gui.py:33–40, 1083–1145; src/gui.py:616–619; src/tiered_gui.py:287–306, 318–447; src/distance_metric.py:14–19, 59–73; src/tiered_outlines.py:14–25, 67–75.

Where to find the implementation

Current line numbers refer to the reviewed local checkout

Topic	File / function / lines
1. Tiers and line picking	src/tiered_gui.py:199–258 — click, assign_segment src/tiered_outlines.py:28–49 — sample_arcs, weighted_error src/tiered_outlines.py:52–128 — checks and acceptance
2. Manual fit and baseline	src/gui.py:621–637, 780–833, 849–857 — controls/model/values src/tiered_gui.py:287–306, 458–479 — reference/config src/uncertainty_gui.py:140–146 — frozen manual RMS
3. Sampling	src/outline_uncertainty.py:101–126 — search_candidates src/historical_sampling.py:57–121 — guided_candidates src/uncertainty_gui.py:58–66, 165–170 — dispatch/batches
4. Mixture and 100% limits	src/uncertainty_gui.py:580–593 — linked percentages/mode src/historical_sampling.py:80–106 — selected rows/retries src/tiered_outlines.py:22–25 — extra-RMS percentage
Scoring geometry and units	src/geometry.py:61–173 — surface and orientation src/search_scoring.py:51–99 — projection/scoring src/distance_metric.py:22–57 — physical conversion
Final distributions and export	src/outline_uncertainty.py:129–141 — summarize src/parameter_histograms.py:48–81 — fractions src/uncertainty_gui.py:1083–1145 — export_result

How the rest of src supports this workflow

event_config.py, images.py and local_fits.py select observations and their metadata. timeseries.py, session.py and paths.py organize times and saved state. outline_preview.py draws accepted silhouettes without adding gates. utils/helioviewer.py fetches data; utils/widgets.py and utils/figure_page.py provide controls and layout; utils/image_display.py changes display limits. example.py and examples/ provide a demo and reference data; __init__.py files mark packages.

Image intensities help the observer trace and manually fit the CME, but the scorer compares geometry through image WCS. Brightness/contrast do not directly change numeric RMS. The reviewed workflow contains no image-noise likelihood, bootstrap, fit Hessian or calibrated posterior weights.

Numerical sanity checks for this explanation confirmed disconnected-arc length weights, the 10%/20%/100% RMS thresholds, zero-baseline behavior, continuous angular summaries, and the historical endpoint/fallback behavior. This document explains the current algorithm; it does not rerun or reinterpret an existing event’s accepted ensemble.

Code locations (paths relative to the project root):

Supporting behavior: src/images.py:118–141; src/utils/image_display.py:49–67; src/outline_preview.py; src/session.py; src/timeseries.py. Source snapshot: b263ea338f19bbd9ff6c953a08f536f1848f53a9.